Petpooja AI Copilot

Full Project Documentation

AI-Powered Revenue Intelligence, Voice Ordering & Inventory Management for Restaurants

A full-stack hackathon project combining FastAPI, React, and locally-running ML models to help restaurant owners make smarter business decisions — with zero cloud AI costs.

Backend	Python 3.12, FastAPI, Uvicorn
Frontend	React 19, Vite 5, Tailwind CSS 4
ML/AI	OpenAI Whisper, RapidFuzz, mlxtend
Data	JSON files (menu, inventory, recipes, orders)

March 7, 2026

Contents

1 Project Overview **3**
- 1.1 Mission Statement . . . 3
- 1.2 System Architecture . . . 3
- 1.3 Technology Stack . . . 4

2 Backend — Module Reference **5**
- 2.1 Module 1 — Revenue Intelligence Engine . . . 5
  - 2.1.1 Menu Matrix Classification . . . 5
  - 2.1.2 Apriori Association Mining . . . 5
  - 2.1.3 API Endpoints — Revenue . . . 6
- 2.2 Module 2 — Voice Ordering Copilot . . . 7
  - 2.2.1 Pipeline Overview . . . 7
  - 2.2.2 Fuzzy Matching Logic . . . 7
  - 2.2.3 API Endpoints — Voice . . . 8
- 2.3 Module 3 — Inventory Management . . . 8
- 2.4 Module 4 — Recipe Management (BOM) . . . 8
- 2.5 Module 5 — Menu CRUD . . . 9

3 Frontend — Component Reference **10**
- 3.1 Design System . . . 10
- 3.2 Page Components . . . 10
  - 3.2.1 LandingPage.jsx — Hero Page . . . 10
  - 3.2.2 Login.jsx — Authentication Screen . . . 10
  - 3.2.3 Setup.jsx — Restaurant Onboarding . . . 10
  - 3.2.4 Dashboard.jsx — Revenue Overview . . . 10
  - 3.2.5 MenuIntelligence.jsx — Menu Matrix Table . . . 11
  - 3.2.6 ComboEngine.jsx — Combo Recommendations . . . 11
  - 3.2.7 VoiceOrders.jsx + VoiceOrderPanel.jsx — Voice Ordering . . . 11
  - 3.2.8 InventoryManager.jsx — Inventory CRUD . . . 11
  - 3.2.9 Settings.jsx — Menu Management & Recipe Builder . . . 12
  - 3.2.10 OrderHistory.jsx — Order Timeline . . . 12
  - 3.2.11 CuratedMenu.jsx & CustomerMenu.jsx — Customer-Facing Menu . . . 12
  - 3.2.12 KitchenDisplay.jsx — Kitchen Display System (KDS) . . . 12

4 Data Schema **13**
- 4.1 menu.json — Menu Item Schema . . . 13
- 4.2 inventory.json — Ingredient Schema . . . 13
- 4.3 recipes.json — Bill of Materials Schema . . . 13
- 4.4 orders.json — Order Record Schema . . . 14

5 Running the Project **15**
- 5.1 Prerequisites . . . 15
- 5.2 Backend Setup . . . 15
- 5.3 Frontend Setup . . . 15
- 5.4 Vite Proxy Configuration . . . 15

5.5 Python Dependencies 16

6 Key Algorithms **17**
6.1 Contribution Margin 17
6.2 Sales Velocity Classification 17
6.3 Menu Matrix 17
6.4 Apriori Association Rules 17
6.5 RapidFuzz Matching Score 17

7 Project File Structure **18**

8 Business Value & Use Cases **20**
8.1 Revenue Intelligence ROI 20
8.2 Voice Ordering ROI 20
8.3 Inventory Management ROI 20
8.4 Target Users 20

9 Future Roadmap **20**

1. Project Overview

1.1 Mission Statement

Petpooja AI Copilot is an AI-powered restaurant management platform that addresses three core pain points for Indian restaurant operators:

1. **Revenue Leakage** — Owners cannot identify which menu items are quietly draining profit and which under-promoted items could be gold mines.
2. **Phone Order Inefficiency** — Staff spends 2–3 hours/day manually taking phone orders, prone to errors, especially with Hinglish-speaking customers.
3. **Inventory Blind Spots** — Ingredient stockouts and waste go undetected until they cause operational crises.

Key Differentiator: All AI/ML models run **100% locally**. No OpenAI API keys. No cloud costs. No customer data leaves the restaurant.

1.2 System Architecture

```
                    React Frontend (Vite)
LandingPage · Login · Setup · Dashboard · MenuIntelligence
ComboEngine · VoiceOrders · Inventory · Settings · KDS

                    REST API (Vite proxy → port 8000)

                FastAPI Backend (Uvicorn)

 Revenue Engine                Voice Copilot
• Margin Analysis            • Whisper STT (local)
• Menu Matrix                • RapidFuzz Matching
• Apriori Mining             • Upsell Engine
• Price Optimizer            • KOT Generator


            Data Layer (db.py | JSON store)
menu.json · inventory.json · recipes.json · orders.json
```

1.3 Technology Stack

Layer	Technology	Purpose
Backend Framework	FastAPI + Uvicorn	REST API server
Language	Python 3.12	Backend logic
ML/AI	OpenAI Whisper (local)	Speech-to-text
NLP Matching	RapidFuzz	Fuzzy order parsing
Association Mining	mlxtend (Apriori)	Combo discovery
Data Analytics	pandas + numpy	Margin/velocity analysis
Frontend Framework	React 19 + Vite 5	SPA web app
Styling	Tailwind CSS 4	Utility-first CSS
Charts	Recharts	Data visualization
Animations	Framer Motion	Page/micro-animations
Icons	Lucide React	Icon system
Data Storage	JSON files	No-database simplicity
API Proxy	Vite dev proxy	Frontend → backend routing

2. Backend — Module Reference

2.1 Module 1 — Revenue Intelligence Engine

revenue_engine.py

File	`backend/modules/revenue_engine.py`
Purpose	Analyzes menu data and transaction history to surface profit opportunities
Key Input	`menu.json` (51 items) + `transactions.json` (800 records)

2.1.1 Menu Matrix Classification

Each item is classified on a 2×2 matrix based on margin (above/below median) and sales velocity (above/below median order count):

	High Velocity	Low Velocity
High Margin	Star	Hidden Star
Low Margin	≈ Plowhorse	× Dog

- **Star** — High margin & high popularity. Maintain and promote heavily.
- **Hidden Star** — High margin but under-ordered. Primary upsell target.
- **Plowhorse** — Sells well but thin margins. Raise price or reduce cost.
- **Dog** — Low margin, low sales. Candidate for removal.

2.1.2 Apriori Association Mining

The engine runs the Apriori algorithm on 800 mock transactions:

Listing 1: Apriori combo discovery core

```python
from mlxtend.frequent_patterns import apriori,
    association_rules

# Build one-hot encoded transaction matrix
te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_array, columns=te.columns_)

# Mine frequent itemsets (min support = 1%)
frequent = apriori(df, min_support=0.01, use_colnames=True)

# Generate association rules (min confidence = 30%)
rules = association_rules(frequent, metric="confidence",
                          min_threshold=0.30)
```

2.1.3 API Endpoints — Revenue

Method	Endpoint	Description
GET	`/menu/analysis`	Full matrix with margin, velocity, classification for all 51 items
GET	`/menu/hidden-stars`	Filtered list of Hidden Star items only
GET	`/menu/combos`	Apriori-mined combo suggestions with AOV lift
GET	`/menu/risks`	Low-margin high-volume risk items
GET	`/menu/price-suggestions`	Rule-based pricing recommendations

2.2 Module 2 — Voice Ordering Copilot

voice_copilot.py

File	`backend/modules/voice_copilot.py`
Purpose	Accepts audio or text input and converts it into a structured kitchen order
Key Libs	`whisper`, `rapidfuzz`, custom upsell + KOT logic

2.2.1 Pipeline Overview

```
Audio File / Typed Text


  [Whisper STT]          ← local model, multilingual
          transcription (str)

  [Hinglish Parser]      ← "ek" → 1, "do" → 2, "teen" → 3
          tokenized words

  [RapidFuzz Matcher]    ← partial ratio  65% to menu items
          matched items + quantities + modifiers

  [Upsell Engine]        ← cross-reference with Apriori combos
          order + upsell suggestions

  [Confirm & KOT]        ← save to orders.json, auto-deplete inventory
          Kitchen Order Ticket (printable text)

  [Stock Impact Report]
```

2.2.2 Fuzzy Matching Logic

Listing 2: RapidFuzz item matching

```python
from rapidfuzz import fuzz, process

def match_item(spoken_token: str, menu_items: list,
               threshold: int = 65) -> dict | None:
    names = [item["name"] for item in menu_items]
    result = process.extractOne(
        spoken_token, names,
        scorer=fuzz.partial_ratio
    )
    if result and result[1] >= threshold:
        matched_name = result[0]
        return next(i for i in menu_items
                    if i["name"] == matched_name)
    return None  # No match above threshold
```

2.2.3 API Endpoints — Voice

Method	Endpoint	Description
POST	`/voice/transcribe`	Accepts multipart audio file; returns Whisper transcription
POST	`/voice/parse-order`	Accepts `{"text": "..."}`; returns structured order JSON
POST	`/voice/confirm-order`	Finalizes order → saves to `orders.json`, depletes inventory, returns KOT
GET	`/voice/orders`	Returns all historical placed orders

2.3 Module 3 — Inventory Management

Inventory CRUD

Data File `backend/data/inventory.json` (20 ingredients)
Purpose Track ingredient stock levels, trigger alerts, manage restocks

Method	Endpoint	Description
GET	`/inventory/items`	All ingredients with stock status
POST	`/inventory/items`	Add new ingredient
PUT	`/inventory/items/{id}`	Update ingredient details
DELETE	`/inventory/items/{id}`	Remove ingredient
PATCH	`/inventory/items/{id}/restock`	Restock by quantity

Each ingredient record contains: `id`, `name`, `unit`, `current_stock`, `min_stock`, `cost_per_unit`, `supplier`, `last_restocked`.

2.4 Module 4 — Recipe Management (BOM)

The Bill of Materials (BOM) system links each menu item to the ingredients it consumes. When an order is confirmed, the system automatically subtracts the required quantities from inventory.

Method	Endpoint	Description
GET	`/recipes`	All recipes keyed by item_id
GET	`/recipes/{item_id}`	BOM for a specific menu item
PUT	`/recipes/{item_id}`	Create or update a recipe

2.5 Module 5 — Menu CRUD

Method	Endpoint	Description
GET	`/menu/items`	All 51 menu items
POST	`/menu/items`	Add new menu item
PUT	`/menu/items/{id}`	Update item details
PATCH	`/menu/items/{id}/toggle`	Toggle availability
DELETE	`/menu/items/{id}`	Remove menu item

3. Frontend — Component Reference

3.1 Design System

All components share a unified design language defined in `index.css`:

- **Dark theme** — `#0F172A` (background) / `#1E293B` (cards)
- **Orange accent** — `#F97415` (primary brand color)
- **Glassmorphism** — `backdrop-filter: blur()` on overlay panels
- **Typography** — Inter via Tailwind's default sans stack
- **Animations** — Framer Motion for page transitions and micro-interactions

3.2 Page Components

3.2.1 LandingPage.jsx — Hero Page

The marketing landing page shown before login. Features:

- Animated hero headline with gradient text
- Feature card grid (Revenue Intelligence, Voice Orders, Inventory)
- Pulsing CTA buttons with hover states
- Stat counters animating from zero using a custom `useCountUp` hook

3.2.2 Login.jsx — Authentication Screen

Restaurant login/signup screen with:

- Email + password form with validation states
- Glassmorphism card on dark gradient background
- Smooth opacity/slide-in entrance animation via Framer Motion
- Error toast on invalid credentials

3.2.3 Setup.jsx — Restaurant Onboarding

Multi-step configuration wizard for new restaurants:

- Restaurant name, type, timezone configuration
- GST rate and base currency settings
- Data seeding options (pre-load demo menu/inventory)

3.2.4 Dashboard.jsx — Revenue Overview

The main home screen after login. Four key widgets:

Widget	Description
Stat Cards (4)	Hidden Stars found, Risk Items, Avg Contribution Margin (%), Orders Today — all with animated count-up
Bar Chart	Top 10 items by contribution margin (), orange gradient bars
Donut Chart	Menu category breakdown — Stars / Hidden Stars / Plowhorses / Dogs
Combo Suggestions	Top 3 AI combos with AOV lift, confidence score, and Promote button

3.2.5 MenuIntelligence.jsx — Menu Matrix Table

Full interactive data table for all 51 menu items:

- **Columns:** Name · Category · Selling Price · Food Cost · Margin (&%) · Velocity · Status
- **Filter pills:** All / Stars / Hidden Stars / Plowhorses / Dogs
- **Search:** Real-time text filtering
- **Sort:** Click any column header to toggle ascending/descending
- Hidden Star rows highlighted with orange left border

3.2.6 ComboEngine.jsx — Combo Recommendations

Displays Apriori-mined bundle suggestions:

- Stats bar: transactions analyzed, associations found, Hidden Star combos
- 2-column grid of combo cards with item chips, confidence %, AOV lift, margin dots
- "Add to Menu" and "Preview" action buttons per card

3.2.7 VoiceOrders.jsx + VoiceOrderPanel.jsx — Voice Ordering

Two-panel voice ordering interface:

Left panel (VoiceOrderPanel):

- Pulsing microphone upload button for WAV/MP3 files
- Hinglish example prompts for quick testing
- Live transcription display with language badge
- Text input fallback for manual order typing

Right panel (VoiceOrders):

- Structured order summary — items, quantities, modifiers, per-item prices
- GST (5%) auto-calculation
- Orange upsell suggestion panel with Accept/Decline
- **Confirm & Send to Kitchen** button
- KOT preview displayed post-confirmation in monospace format

3.2.8 InventoryManager.jsx — Inventory CRUD

Full CRUD interface for the 20 inventory ingredients:

- Stock status bar (Green/Yellow/Red) per ingredient
- Low-stock alert badges with configurable thresholds
- Edit modal with inline form for all fields
- Restock modal for quick quantity top-up
- Supplier info and last restock date displayed per row

3.2.9 Settings.jsx — Menu Management & Recipe Builder

Combined menu CRUD and recipe (BOM) editor:

- Full menu table with Add / Edit / Delete / Toggle availability
- Per-item recipe builder — add ingredients with quantities and units
- Modal form with validation for menu item fields
- Availability toggle shows real-time status badge

3.2.10 OrderHistory.jsx — Order Timeline

Chronological log of all voice-processed orders:

- Time filters: Today / This Week / All Time
- Timeline cards with Order ID, timestamp, item summary, total (), language badge
- Click-to-expand for full item breakdown with quantities, modifiers, per-item prices

3.2.11 CuratedMenu.jsx & CustomerMenu.jsx — Customer-Facing Menu

Public-facing menu for dine-in or QR-code ordering:

- Category filter pills and live search
- Item cards with name, description, price, and availability indicator
- Add-to-cart with quantity stepper
- Cart drawer with item list, subtotal, GST, and checkout CTA

3.2.12 KitchenDisplay.jsx — Kitchen Display System (KDS)

Real-time display for kitchen staff:

- Order queue showing all active orders
- Status transitions: Pending → In Progress → Ready → Served
- Visual color coding per status
- Elapsed time counter per order ticket
- Priority indicators for urgent orders

4. Data Schema

4.1 menu.json — Menu Item Schema

Listing 3: Single menu item record

```
{
  "item_id":       "M001",
  "name":          "Classic Veg Burger",
  "category":      "Burgers",
  "selling_price": 149.0,
  "food_cost":      52.0,
  "available":     true,
  "description":   "Crispy veggie patty with lettuce and sauce"
}
```

4.2 inventory.json — Ingredient Schema

Listing 4: Single inventory record

```
{
  "id":              "I001",
  "name":            "Refined Flour",
  "unit":            "kg",
  "current_stock":   12.5,
  "min_stock":       5.0,
  "cost_per_unit":   45.0,
  "supplier":        "Metro Cash & Carry",
  "last_restocked": "2026-03-01"
}
```

4.3 recipes.json — Bill of Materials Schema

Listing 5: BOM record for one menu item

```
{
  "M001": {
    "item_name": "Classic Veg Burger",
    "ingredients": [
      { "ingredient_id": "I001", "name": "Refined Flour",
        "qty": 0.1, "unit": "kg" },
      { "ingredient_id": "I005", "name": "Patty Mix",
        "qty": 0.08, "unit": "kg" }
    ]
  }
}
```

4.4 orders.json — Order Record Schema

Listing 6: Confirmed order record

```
{
  "order_id":   "ORD-20260307-001",
  "timestamp":  "2026-03-07T12:00:00",
  "source":     "voice",
  "language":   "hinglish",
  "items": [
    {
      "item_id":   "M001",
      "name":      "Classic Veg Burger",
      "qty":       2,
      "modifiers": ["extra cheese"],
      "price":     149.0
    }
  ],
  "subtotal":   298.0,
  "gst":        14.9,
  "total":      312.9,
  "status":     "confirmed",
  "kot":        "=== KITCHEN ORDER TICKET ===\n..."
}
```

5. Running the Project

5.1 Prerequisites

- Python 3.10+ (recommended: 3.12)
- Node.js 18+ and npm 9+
- ~4 GB disk space for Whisper base model (auto-downloaded on first use)
- Recommended: 8 GB RAM (Whisper base model uses ~1 GB)

5.2 Backend Setup

Listing 7: Backend startup commands

```
# Navigate to backend
cd f:\Petpooja\backend

# Install Python dependencies
pip install -r requirements.txt

# Generate mock transaction data (run ONCE)
python generate_transactions.py

# Start the API server
python -m uvicorn main:app --host 0.0.0.0 --port 8000

# Server is now live at: http://localhost:8000
# API docs (Swagger UI) at: http://localhost:8000/docs
```

5.3 Frontend Setup

Listing 8: Frontend startup commands

```
# Navigate to frontend
cd f:\Petpooja\frontend

# Install Node dependencies
npm install

# Start Vite development server
npm run dev

# App is now live at: http://localhost:5173
# (auto-proxies /api/* requests to localhost:8000)
```

5.4 Vite Proxy Configuration

The `vite.config.js` configures the dev server to proxy all `/api` requests to the backend:

Listing 9: vite.config.js proxy setup

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8000',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, '')
      }
    }
  }
})
```

5.5 Python Dependencies

Listing 10: requirements.txt

```
fastapi
uvicorn
pandas
numpy
mlxtend
rapidfuzz
openai-whisper
python-multipart
```

6. Key Algorithms

6.1 Contribution Margin

$$\text{Contribution Margin} = \text{Selling Price} - \text{Food Cost}$$

$$\text{Margin } \% = \frac{\text{Selling Price} - \text{Food Cost}}{\text{Selling Price}} \times 100$$

Items are classified as *high margin* if their margin % exceeds the median across all items.

6.2 Sales Velocity Classification

Order frequency per item is extracted from `transactions.json`. Items above the median order count are classified as **Fast Mover**; below median as **Slow Seller**; within 10% of median as **Moderate**.

6.3 Menu Matrix

$$\text{Class}(i) = \begin{cases} \text{Star} & \text{if margin}(i) > m_{50} \ \wedge \ \text{velocity}(i) > v_{50} \\ \text{Hidden Star} & \text{if margin}(i) > m_{50} \ \wedge \ \text{velocity}(i) \leq v_{50} \\ \text{Plowhorse} & \text{if margin}(i) \leq m_{50} \ \wedge \ \text{velocity}(i) > v_{50} \\ \text{Dog} & \text{otherwise} \end{cases}$$

where m_{50} and v_{50} are the median margin % and median order count respectively.

6.4 Apriori Association Rules

Given a set of transactions $T = \{t_1, t_2, \ldots, t_n\}$ and itemset X:

$$\text{Support}(X) = \frac{|\{t \in T : X \subseteq t\}|}{|T|}$$

$$\text{Confidence}(X \Rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

The system mines rules with **minimum support = 1%** and **minimum confidence = 30%** from the 800 transaction dataset.

6.5 RapidFuzz Matching Score

For spoken token q against menu item name s:

$$\text{score}(q, s) = \text{PartialRatio}(q, s) \in [0, 100]$$

A match is accepted only if $\text{score}(q, s) \geq 65$. The item with the highest score above threshold is selected.

7. Project File Structure

```
Petpooja/
 backend/
    main.py                      # FastAPI app entry | all route definitions
    supplier_server.py           # Supplier-facing secondary server
    generate_transactions.py     # One-time mock data generator
    requirements.txt             # Python package list
    .env                         # Environment secrets (not committed)
    data/
       menu.json                # 51 menu items with pricing & categories
       inventory.json           # 20 ingredients with stock levels
       recipes.json             # BOM for all 51 items (auto-depletion)
       orders.json              # Runtime: all confirmed orders
       transactions.json        # 800 historical mock PoS transactions
    modules/
        __init__.py
        db.py                    # JSON load/save helpers (data access layer)
        revenue_engine.py        # Margin, velocity, matrix, Apriori, pricing
        voice_copilot.py         # Whisper STT, fuzzy match, upsell, KOT

 frontend/
    index.html                   # Vite HTML entry point
    vite.config.js               # Vite + API proxy configuration
    package.json                 # npm scripts and dependencies
    eslint.config.js
    public/
       petpoja.png               # App logo used in sidebar and favicon
    src/
        main.jsx                 # React app mount point
        App.jsx                  # Root component: sidebar + routing logic
        App.css
        index.css                # Global design system & Tailwind overrides
        utils/
           apiFetch.js          # Centralized fetch wrapper with error handling
        components/
            LandingPage.jsx      # Hero/marketing landing page
            Login.jsx            # Login & authentication screen
            Setup.jsx            # Restaurant onboarding wizard
            Sidebar.jsx          # Navigation sidebar with route highlighting
            Dashboard.jsx        # Revenue overview | stats, charts, combos
            MenuIntelligence.jsx # Sortable/filterable menu matrix table
            ComboEngine.jsx      # Apriori combo cards (main page)
            ComboSuggestions.jsx # Reusable combo suggestion sub-component
            RevenueTable.jsx     # Detailed revenue breakdown sub-component
            VoiceOrders.jsx      # Full voice ordering page
            VoiceOrderPanel.jsx  # Audio upload + transcription panel
```

```
            OrderHistory.jsx     # Order log timeline with expand/collapse
            InventoryManager.jsx # Inventory CRUD + stock alerts + restock
            Settings.jsx         # Menu management + recipe BOM builder
            CuratedMenu.jsx      # Customer-facing browsable menu
            CustomerMenu.jsx     # Full customer ordering experience + cart
            KitchenDisplay.jsx   # Real-time KDS for kitchen staff

docs/
   Petpooja_Project.tex         # This document (full project documentation)
   Petpooja_ML_Models.tex       # Proposed ML model implementation guide
   Petpooja_ML_Models.pdf       # Compiled ML guide
   PROJECT_EXPLANATION.md       # Page-by-page explanation (markdown)
   Petpooja Track *.pdf        # Hackathon problem statement

README.md                        # Quick-start and overview guide
```

8. Business Value & Use Cases

8.1 Revenue Intelligence ROI

A typical restaurant with 50 menu items may have 10–15 Hidden Stars. Promoting these high-margin items to same sales level as Stars could represent a 15–25% increase in overall contribution margin — without adding a single new item or customer.

Example: "Paneer Makhani Pizza" has a 69.9% margin but orders 40% less than average. Featuring it on the menu front page or bundling it in a combo could add 8,000–12,000/month in additional margin.

8.2 Voice Ordering ROI

Metric	Manual	With Copilot
Phone orders/day	50	50
Avg. time per order	3 min	45 sec
Staff time/day on orders	150 min	37 min
Order error rate	~8%	~2%
Languages supported	1	3+ (Hinglish)

8.3 Inventory Management ROI

Auto-depletion on confirmed orders means inventory is always accurate without manual stock-takes. Low-stock alerts prevent costly mid-service stockouts. Supplier info on every ingredient enables one-click restock workflows.

8.4 Target Users

- **Restaurant Owner** — Dashboard, Revenue Intelligence, Combo Engine
- **Front-of-House Staff** — Voice Orders, Customer Menu
- **Kitchen Staff** — Kitchen Display System
- **Restaurant Manager** — Inventory Manager, Settings, Order History

9. Future Roadmap

1. **Phase 1 — Demand Forecasting:** Implement Facebook Prophet to predict per-item sales for the next 7 days. Feed predictions into automatic purchase order generation.
2. **Phase 2 — NLP Intent Parser:** Replace RapidFuzz with a local Gemma 2B LLM for context-aware order parsing. Handle corrections, references, and complex modifier instructions.

3. **Phase 3 — Collaborative Filtering:** Add customer identity via phone number / loyalty card. Apply SVD matrix factorization to personalize upsell recommendations per customer.
4. **Phase 4 — Anomaly Detection:** Deploy Isolation Forest on hourly operations data to flag revenue drops, unusual void rates, or inventory theft automatically.
5. **Phase 5 — Mobile App:** React Native wrapper for the KDS and voice order panel so kitchen staff and floor staff can use mobile devices.

See `docs/Petpooja_ML_Models.pdf` for full technical implementation guides for all proposed ML models.